

Taller: Control de Calidad de Software

Clase 12 - Herramientas para soporte de pruebas





- Actividades de la prueba
- Objetivos de las herramientas de pruebas
- Clasificación de herramientas
- Automatización de pruebas
 - Beneficios y Riesgos
 - Herramientas: como elegir
- Factores de éxito en la selección de herramientas

- **Planificación:** se definen los objetivos y el enfoque para cumplirlos.
- **Seguimiento y Control:** comparación continua del avance real vs plan; tomar medidas necesarias para cumplir el objetivo. Informes de avance.
- **Análisis:** se identifican las características del objeto de prueba: “que probar”.
- **Diseño:** se definen casos de prueba y sets: “como probar”
- **Implementación:** se desarrollan procedimientos, se organiza un calendario de ejecución, se construyen entornos y se preparan datos de prueba.
- **Ejecución:** se ejecutan las suites de prueba siguiendo el calendario.
- **Compleción:** se recopilan datos de las actividades completadas: informes de defectos, analizar lecciones aprendidas e identificar mejoras.

Se usan para apoyar distintas actividades de la prueba

01	Uso directo en la prueba	• Ejecución • Preparación de datos • Configuraciones
02	Gestión en general	• Requisitos, casos de prueba • Procedimientos de prueba, Scripts de automatización • Resultados, defectos, informes, monitorización
03	Investigación y evaluación	• Frameworks de automation, herramientas de test management
04	Cualquier herramienta que asista la prueba	• Planilla de cálculo, ¿por qué no?

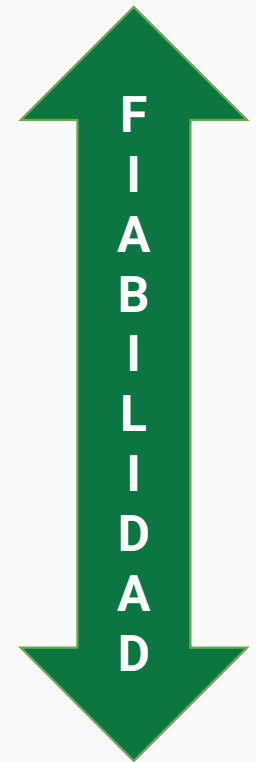
Herramientas de prueba: Objetivos

Mejorar eficiencia de actividades de prueba manuales en proceso de prueba

Mejorar eficiencia de las actividades de prueba: automatizar

Alternativas a pruebas manuales no viables: automatizar

Mejorar la calidad en general en actividades de prueba: Consistencia de información y reproductibilidad de defectos.





- Una sola actividad
- Varias actividades

Son intrusivas??
(afectan el resultado
de las pruebas)



Soporte para Gestión de la Prueba y Productos de Prueba

Cubren el ciclo completo de desarrollo de Software

- Gestión del ciclo de vida
- Requisitos
- Defectos
- Configuración
- IC



Soporte para Gestión de la Prueba y Productos de Prueba

	Cualidad 1	Cualidad 2
QMERTY	Gestión de Pruebas completa (TCMS) Es una plataforma especializada para la gestión del ciclo de vida de las pruebas (manuales, exploratorias y automatizadas), ofreciendo visibilidad y trazabilidad.	Integración con Automatización y DevOps Permite integrar y ejecutar scripts de pruebas automatizadas y reportar los resultados directamente, lo que facilita el flujo CI/CD (Integración/Entrega Continua).
JIRA	Gestión Ágil de Proyectos Es líder en el seguimiento de historias de usuario, tareas y sprints, esencial para equipos que trabajan con metodologías Scrum o Kanban.	Flujos de Trabajo Altamente Personalizables Permite definir y automatizar flujos de trabajo (workflows) específicos para diferentes tipos de incidencias (requisitos, tareas, bugs), adaptándose a las necesidades exactas del equipo.
GIT	Control de Versiones Distribuido Permite a múltiples desarrolladores y testers trabajar en el mismo código de forma segura y paralela, manteniendo un historial detallado de todos los cambios.	Soporte para Ramificación (Branching) Flexible Facilita la creación de ramas aisladas (features, hotfixes, releases), permitiendo la experimentación y el desarrollo sin afectar el código principal hasta que los cambios estén listos.
JENKINS	Motor de Integración Continua (CI) Automatiza la compilación, prueba y empaquetado de software cada vez que se realiza un cambio en el código fuente, proporcionando feedback rápido sobre la calidad.	Ecosistema Extenso de Plugins Su gran cantidad de plugins permite integrarse con prácticamente cualquier herramienta (GIT, JIRA, Selenium, bases de datos, servicios en la nube), haciéndolo muy versátil y configurable.
MANTIS	Sistema de Seguimiento de Errores Simple (Bug Tracker) Es una herramienta web de código abierto y fácil de usar (generalmente gratuita), ideal para rastrear y gestionar los defectos (bugs) del software.	Personalización de Roles y Permisos Permite definir diferentes niveles de acceso y roles de usuario (reportero, desarrollador, gerente, etc.) para controlar quién puede ver y modificar las incidencias en el proyecto.

Soporte para Diseño e Implementación de Pruebas

Ayudan a crear productos de trabajo mantenibles como:

- Casos de Prueba
- procedimientos de prueba
- datos de prueba



Algunas herramientas son:

- Para diseño de pruebas
- Para Prueba basada en modelos
- Para Preparación de datos de prueba
- Para implementar ATDD/BDD/TDD



Ejecución y registro
Integración con automatizaciones



Soporte para Diseño e Implementación de Pruebas

	Cualidad 1	Cualidad 2
MOCKITO	Aislamiento de Dependencias (Mocking) Permite crear objetos simulados (mocks) que imitan el comportamiento de clases dependientes (como bases de datos, servicios web o componentes externos). Esto asegura que las pruebas unitarias se centren exclusivamente en la lógica de la clase bajo prueba.	API Clara y Expresiva Utiliza una sintaxis fluida y un lenguaje intuitivo (como <code>when().thenReturn()</code> y <code>verify()</code>) que hace que las pruebas sean más fáciles de leer, entender y mantener que con otros frameworks de mocking.
BDD CUCUMBER	Lenguaje Común (Gherkin) Utiliza el lenguaje Gherkin (con palabras clave como Given, When, Then) para describir escenarios de prueba en un formato de lenguaje natural. Esto facilita la colaboración y el entendimiento entre desarrolladores, testers y el equipo de negocio (no técnicos).	Enfoque en el Comportamiento (Behavior) Las pruebas se centran en el comportamiento esperado del usuario (la Historia de Usuario) en lugar de los detalles técnicos. Esto garantiza que la automatización esté directamente alineada con los requisitos del negocio.
TESTRAIL	Repositorio Centralizado de Pruebas Proporciona un único lugar para organizar, almacenar y gestionar todos los casos de prueba (manuales y automatizados), planes de prueba y ejecuciones, haciendo que la gestión del QA sea escalable.	Visibilidad en Tiempo Real y Trazabilidad Ofrece paneles de control e informes potentes que muestran el progreso de las pruebas, la cobertura de requisitos y el estado de calidad en tiempo real, vinculando directamente las pruebas a los defectos y requisitos (trazabilidad).

Soporte para Pruebas Estáticas

Dan apoyo a:

- Revisiones
- Análisis estático



Soporte para Pruebas Estáticas

	Cualidad 1	Cualidad 2
SONARQUBE	Definición y Aplicación de Quality Gates (Puertas de Calidad) Permite definir un conjunto de condiciones obligatorias (Quality Gates) que el código debe cumplir (ej. cobertura de pruebas mínima, cero bugs críticos). Si el código no pasa la Gate, el proceso de Integración Continua (CI) se detiene, previniendo que código de mala calidad o inseguro llegue a producción.	Análisis Estático y Continuo del Código (SAST) Analiza el código fuente sin necesidad de ejecutarlo (Análisis Estático) e identifica proactivamente bugs, vulnerabilidades de seguridad (hotspots), olores de código, y problemas de mantenibilidad en las primeras etapas del ciclo de desarrollo.

Soporte para Ejecución y Registro de pruebas

Herramientas para:

- Creación de scripts automatizados
- Ejecución de pruebas automatizadas
- Arneses de prueba
- creación y ejecución de pruebas unitarias



Soporte para Ejecución y Registro de pruebas

	Cualidad 1	Cualidad 2
SE Selenium	Soporte Cross-Browser y Multi-Lenguaje Permite ejecutar pruebas de manera consistente en la mayoría de los navegadores principales (Chrome, Firefox, Safari, Edge) y es compatible con múltiples lenguajes de programación (Java, Python, C#, etc.), ofreciendo flexibilidad al equipo.	Automatización de Interacciones Web Proporciona una API (interfaz de programación) que simula las acciones de un usuario real (clics, escritura, navegación) directamente en la página web, siendo el estándar de facto para la automatización frontend.
APPIUM	Reutilización de Código entre Plataformas Permite escribir pruebas para aplicaciones nativas, híbridas y web móviles utilizando la misma API y lógica para iOS y Android, maximizando la reutilización de código y esfuerzos de automatización.	No Requiere Modificar la Aplicación Utiliza frameworks de automatización estándar de cada sistema operativo (como UI Automator 2 para Android y XCUITest para iOS) y no requiere que se modifique el código fuente ni se añadan bibliotecas específicas a la aplicación.
JUNIT	Adopción Amplia y Código Abierto Es el framework más utilizado y estándar en el ecosistema Java para escribir y ejecutar pruebas unitarias. Al ser de código abierto, cuenta con una vasta documentación, gran comunidad de apoyo e integración con casi todas las herramientas de desarrollo.	Anotaciones Simples para la Ejecución Proporciona una estructura clara mediante anotaciones (@Test, @BeforeEach, @AfterAll) para definir los casos de prueba, y facilita la generación de reportes automáticos que indican el estado de las pruebas (pasaron/fallaron).
JACOCO (Java Code Coverage)	Medición de la Cobertura del Código Mide la efectividad de las pruebas al calcular qué porcentaje y qué líneas específicas del código fuente fueron ejecutadas por las pruebas unitarias y de integración, ayudando a identificar áreas no probadas.	Integración con Herramientas de Construcción Se integra fácilmente con herramientas populares de construcción como Maven y Gradle para automatizar la generación de informes detallados (XML y HTML) que pueden ser visualizados en plataformas como SonarQube.
Python	Fácil de Aprender y Usar (Sintaxis Clara) Posee una sintaxis limpia y altamente legible que utiliza sangría en lugar de llaves, asemejándose al inglés simple. Esto reduce la curva de aprendizaje y facilita la mantención de los scripts de automatización.	Amplia Librería Estándar y Módulos Cuenta con una vasta biblioteca de módulos (frameworks como Selenium, unittest, pytest) que le permite ser utilizado en diversos dominios: desarrollo web, automatización de tareas (scripting), ciencia de datos e inteligencia artificial.

Soporte para Medición del Rendimiento y Análisis dinámico

Dan soporte a pruebas no funcionales, herramientas para:

- Pruebas de estrés, carga y concurrencia
- Monitoreo de procesos
- Análisis dinámico
- creación y ejecución de pruebas unitarias



Herramientas de prueba: Clasificación

	Cualidad 1	Cualidad 2
GATLING	Arquitectura Asíncrona (Akka) Utiliza una arquitectura asíncrona basada en Akka, lo que le permite simular un gran número de usuarios virtuales con un consumo mínimo de recursos del sistema. Esto la hace muy eficiente para pruebas de carga a gran escala.	Dominio Específico del Lenguaje (DSL) Los scripts de prueba se escriben como código Scala (o lenguajes modernos como Python), utilizando un DSL intuitivo. Este enfoque es preferido por los desarrolladores y facilita la mantenibilidad, legibilidad y la integración en flujos CI/CD.
GRAFANA	Creación de Dashboards Interactivos Proporciona una interfaz rica para crear paneles de control (dashboards) de manera intuitiva, que combinan múltiples visualizaciones (gráficos, medidores, tablas). Esto permite el monitoreo en tiempo real de métricas y rendimiento.	Soporte de Múltiples Fuentes de Datos No está ligada a una única fuente de datos. Puede conectarse y extraer métricas de una amplia variedad de fuentes (Prometheus, Elasticsearch, bases de datos SQL/NoSQL, InfluxDB), centralizando la información de diferentes sistemas.
JMETER	Pruebas de Carga Multi-Protocolo Es una herramienta extremadamente robusta y flexible que no se limita a pruebas web. Permite realizar pruebas de carga y rendimiento en una gran cantidad de protocolos, incluyendo HTTP/S, FTP, SOAP/REST (servicios web), JDBC (bases de datos) y más.	Fácil Creación de Escenarios (Diseño Gráfico) Ofrece una interfaz gráfica de usuario completa que facilita la creación visual de planes de prueba, thread groups (grupos de usuarios) y listeners (elementos de reporte), haciéndola accesible a usuarios con menos experiencia en programación.
KIBANA	Interfaz de Búsqueda Interactiva (Discover) Proporciona la funcionalidad Discover, una potente interfaz que permite a los usuarios buscar, filtrar y explorar grandes volúmenes de datos de log y eventos almacenados en Elasticsearch de forma rápida e interactiva.	Componente Clave del Stack ELK/Elastic Es la capa de presentación del Elastic Stack (Elasticsearch, Logstash, Kibana). Su principal propósito es la visualización y análisis de datos de series temporales y logs, lo que es crucial para la observabilidad y el diagnóstico de errores.

Soporte para Necesidades de Pruebas Especializadas

Herramientas para:

- Evaluación de la calidad de datos
- Conversión y migración de datos
- Usabilidad
- Accesibilidad
- Seguridad
- Portabilidad (mismo software en varias plataformas compatibles)



Beneficios:

- Reducción del trabajo manual
- Mayor Consistencia y Repetibilidad
- Evaluación Objetiva
- Acceso rápido a información y reportes de la prueba

Riesgos en procesos de adopción de herramientas:

- Expectativas poco realistas
- Relación tiempo/costo/esfuerzo subestimado
- Relación tiempo/costo/esfuerzo para obtener beneficios subestimado
- Esfuerzo para mantener activos de prueba subestimado
- Demasiada confianza en las herramientas

Riesgos en procesos ejecutándose:

- Sin control de versiones
- Hay que entender relación entre herramientas críticas para interoperabilidad.
- Proveedor de una herramienta deja de mantenerla, o la saca del mercado.
- Herramientas Open Source sin soporte
- Herramienta no soportada por nuevas tecnologías

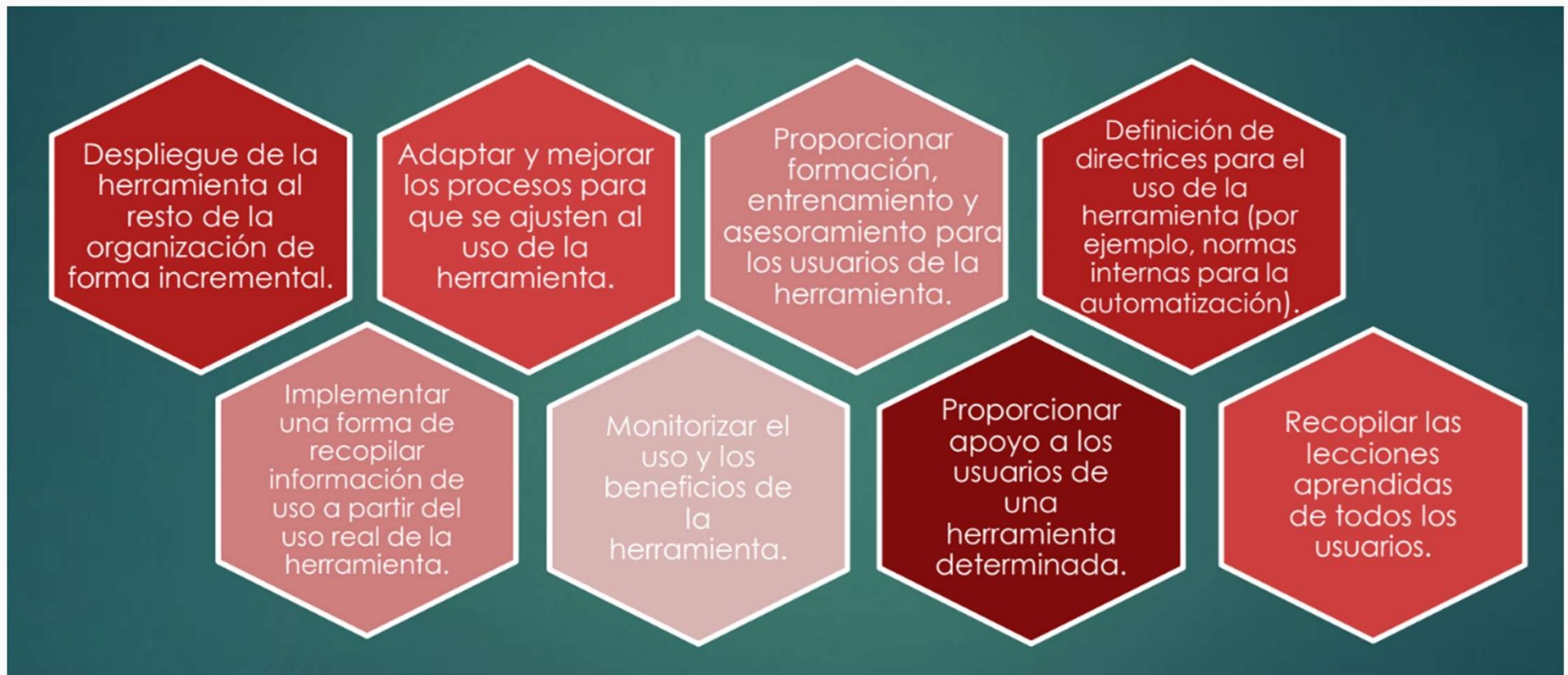
Principios para selección de herramientas:

- Madurez de la organización
- Identificar oportunidades de mejora
- Comprender las tecnologías utilizadas por el objeto de prueba
- Asegurar la compatibilidad e integración entre herramientas
- Contemplar períodos de prueba gratuitos
- Evaluar al proveedor
- Identificar requisitos para entrenamiento y asesoramiento
- Evaluar modelos de licenciamiento

Pruebas de Concepto (poc) o proyectos piloto

- Obtener conocimiento de la herramientas, identificar fortalezas y debilidades
- Definir proceso de uso de la herramienta, y definición de estándares
- Evaluar relación costo-beneficio
- Entender las métricas que ofrece la herramienta, o las que se pueden obtener de la misma.

Selección de herramientas: Factores de éxito



¿Preguntas?



GRACiAS

